

Apprendimento Non Supervisionato tramite Algebra Lineare

Metodi Statistici e Matematici per il Machine Learning

Traduzione italiana — materiale originale a cura di Davide Evangelista

Nei capitoli precedenti, il nostro paradigma di apprendimento si basava sulla **supervisione**: ci venivano forniti coppie ingresso–uscita (X, Y) , e l’obiettivo era *apprendere* una funzione che mapasse ogni ingresso nell’uscita corretta. Abbiamo sviluppato modelli come la regressione lineare, la regressione logistica e semplici reti neurali, allenandoli tramite metodi iterativi di **ottimizzazione basata sul gradiente** come *SGD* e *Adam*. In quei contesti, la funzione di perdita era esplicitamente definita confrontando le predizioni con le etichette vere, e apprendere significava ridurre progressivamente questa discrepanza.

Ci rivolgiamo ora a una prospettiva diversa, ma altrettanto fondamentale: l’**apprendimento non supervisionato**. In questo contesto, ci vengono forniti solo i dati X , senza alcun valore target Y associato. Il compito è scoprire la **struttura** nascosta nei dati, come pattern, raggruppamenti, rappresentazioni a bassa dimensionalità, o feature significative che non sono fornite esplicitamente. Invece di apprendere “dalle risposte”, ora cerchiamo di apprendere **dai dati stessi**.

L’apprendimento non supervisionato gioca un ruolo cruciale nel Machine Learning e nell’analisi dati moderni. Due delle sue aree di maggiore impatto sono:

- **Riduzione della dimensionalità / compressione dei dati**: i dataset del mondo reale vivono spesso in spazi ad alta dimensionalità, ma gran parte di quella dimensionalità è ridondante. Trovando rappresentazioni compatte e informative, possiamo migliorare l’immagazzinamento, la visualizzazione, e persino le prestazioni dei modelli supervisionati. *Esempio: ridurre il numero di feature necessarie per descrivere immagini, registrazioni audio, o embedding testuali.*
- **Clustering**: identificare gruppi o categorie nei dati *senza* etichette. *Esempio: segmentare i clienti in base al comportamento d’acquisto, raggruppare immagini simili senza annotazioni, riconoscere pattern in dati biologici o di sensori.*

A differenza del training delle reti neurali, dove gradienti e learning rate giocano un ruolo centrale, molti potenti metodi non supervisionati si basano interamente sull’**algebra lineare**. Uno strumento particolarmente elegante in questa direzione è la **Decomposizione ai Valori Singolari** (*Singular Value Decomposition*, SVD), una tecnica fondamentale di fattorizzazione di matrici che rivela la struttura geometrica e statistica dei dati. Con la SVD, possiamo:

- scoprire le direzioni principali di variazione nei dati,
- costruire approssimazioni ottimali a rango basso (per la compressione),
- e applicare queste idee per eseguire clustering basato sulla struttura latente.

In questo capitolo, studieremo come l’apprendimento non supervisionato possa essere realizzato **senza gradienti**, usando solo decomposizioni di matrici e ragionamento algebrico-lineare. Vedremo che, sebbene filosoficamente diversi dalle tecniche supervisionate studiate finora, questi metodi condividono la stessa motivazione: estrarre informazione significativa dai dati. L’unica differenza, rispetto all’apprendimento supervisionato, è che ora vogliamo farlo **senza mai conoscere in anticipo le “risposte corrette”**.

Iniziamo introducendo la SVD, capendo come si relaziona alla massimizzazione della varianza e alle proiezioni ottimali, e poi mostriamo come possa essere usata per la riduzione della dimensionalità e il clustering.

1 La Decomposizione ai Valori Singolari (SVD)

Nell'analisi dei dati, è spesso necessario comprimere i dati, sia per renderli più gestibili sia per poter visualizzare le feature più importanti, riducendo la ridondanza. I due compiti, di solito chiamati **compressione dei dati** e **riduzione della dimensionalità**, sono matematicamente equivalenti e strettamente legati al concetto di **Decomposizione ai Valori Singolari (SVD)** di una matrice.

Ricordiamo che:

Nota

Una matrice invertibile $A \in \mathbb{R}^{n \times n}$ si dice **ortogonale** se $A^T A = I$ o, equivalentemente, $A^T = A^{-1}$.

Ora, consideriamo una matrice data $A \in \mathbb{R}^{m \times n}$. Si può dimostrare che essa può **sempre** essere fattorizzata nel prodotto di tre matrici,

$$A = U \Sigma V^T$$

dove $U \in \mathbb{R}^{m \times m}$ e $V \in \mathbb{R}^{n \times n}$ sono matrici ortogonali, mentre $\Sigma \in \mathbb{R}^{m \times n}$ è una matrice rettangolare non nulla solo sulla diagonale. Tale decomposizione è chiamata **Decomposizione ai Valori Singolari (SVD)** di A .

Di particolare interesse nella nostra analisi sono i valori sulla diagonale di Σ , chiamati **valori singolari** di A , e solitamente indicati con $\sigma_1, \dots, \sigma_{\min\{m,n\}}$. In particolare, è noto che i valori singolari:

- sono sempre maggiori o uguali a 0, cioè $\sigma_i \geq 0, \forall i$;
- sono ordinati in ordine decrescente, cioè $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$;
- possono essere usati per determinare il rango di A , dato che esso è uguale al numero di valori singolari strettamente maggiori di zero, cioè se $\sigma_1 \geq \sigma_2 \geq \dots \sigma_r > 0$ e $\sigma_{r+1} = 0$ per un certo indice r , allora $r = rk(A)$.

Una proprietà utile della SVD di A è che può essere usata per comprimere le informazioni contenute in A stessa. Infatti, si noti che la decomposizione SVD permette di riscrivere A come somma di matrici semplici, cioè

$$A = U \Sigma V^T = \sum_{i=1}^r \sigma_i u_i v_i^T$$

dove u_i e v_i sono rispettivamente le colonne di U e V . Ogni termine $u_i v_i^T$ è una matrice di rango 1 chiamata **diade** (*dyad*), e l' i -esimo valore singolare σ_i rappresenta l'importanza dell' i -esima diade nella costruzione di A . In particolare, la decomposizione SVD permette di scomporre A nella somma di matrici con contenuto informativo decrescente.

La decomposizione SVD può essere usata per comprimere la matrice A considerando la sua approssimazione di rango k , A_k , definita come

$$A_k = \sum_{i=1}^k \sigma_i u_i v_i^T.$$

Nota

L'approssimazione di rango k , A_k , di A si ottiene semplicemente fermando la somma che definisce A tramite diadi al termine $k \leq r$. Quando $k = r$, chiaramente $A_k = A$.

Si può inoltre dimostrare che l'approssimazione di rango k di A è la matrice di rango k che minimizza la distanza (espressa in norma 2) da A , cioè

$$A_k = \arg \min_{M: rk(M)=k} ||M - A||_2.$$

1.1 SVD di una matrice in Python

La SVD può essere facilmente calcolata in Python usando le funzioni del pacchetto `numpy`.

```
# Importa numpy
import numpy as np

# Consideriamo una matrice di esempio
A = np.array([
    [-1, -2, 0, 1, -2, -3],
    [-1, -2, -3, -2, 0, -3],
    [-1, -3, 1, 3, 2, -4],
    [2, 1, -1, 0, -2, 3],
    [0, -3, -1, 2, -1, -3],
    [1, -3, 2, 6, 0, -2],
    [-3, 1, 0, -4, 2, -2],
    [-2, 2, -2, -6, -2, 0],
    [-3, -1, 2, 0, 2, -4],
    [2, -2, 0, 4, -1, 0],
])

# Misura la forma di A: qual e' il rango massimo?
m, n = A.shape
print(f"La forma di A e': {(m, n)}.")

# Calcola la decomposizione SVD di A e verifica le forme
U, s, VT = np.linalg.svd(A, full_matrices=True)
print(U.shape, s.shape, VT.shape)

# Definisce la matrice completa S
S = np.zeros((m, n))
S[:n, :n] = np.diag(s)
```

```
La forma di A e': (10, 6).
(10, 10) (6,) (6, 6)
```

Qui abbiamo calcolato la SVD di A tramite la funzione `np.linalg.svd()`, che prende in ingresso una matrice e restituisce una tripla U , s , VT , che rappresentano le matrici U , V^T e una versione **vettorizzata** di Σ che contiene solo la diagonale (per risparmiare memoria!).

Si noti che, in alcune situazioni, può essere utile calcolare la matrice completa Σ , come fatto in fondo allo snippet di codice sopra.

Attenzione

Se la matrice completa Σ può essere evitata, non costruirla esplicitamente.

Esercizio

Verifica che l'algoritmo sopra funzioni come previsto, dimostrando che $A \approx U\Sigma V^T \iff \|A - U\Sigma V^T\|_2 \approx 0$.

Esercizio

Calcola l'approssimazione di rango k , A_k , di A . Nello specifico, scrivi una funzione Python che prenda in ingresso un intero $k \leq \min\{m, n\}$ e calcoli l'approssimazione di rango k , $A_k = \sum_{i=1}^k \sigma_i u_i v_i^T$. Poi, testala su una matrice a tua scelta e calcola l'errore di approssimazione in norma di Frobenius, $\|A - A_k\|_F$.

1.2 Il rango numerico di A

In teoria, il rango di una matrice è il numero di valori singolari non nulli nella sua Decomposizione ai Valori Singolari (SVD). Tuttavia, nei dati del mondo reale (e nell'aritmetica in virgola mobile), i valori singolari raramente si annullano esattamente; piuttosto, molti diventano *molto piccoli* a causa di rumore, correlazioni, o precisione computazionale. Il **rango numerico** di una matrice si riferisce al numero di valori singolari significativamente maggiori di zero secondo una tolleranza scelta. In pratica, rappresenta la *dimensionalità effettiva* della matrice: quante direzioni nello spazio definito da A contengono realmente informazione. I valori singolari piccoli sono spesso associati a rumore o struttura ridondante, mentre i valori singolari dominanti catturano pattern significativi. Identificare il rango numerico è fondamentale per compiti come la riduzione della dimensionalità, la compressione e il filtraggio del rumore, poiché ci dice quante componenti principali servano realmente per rappresentare fedelmente il dataset.

Esercizio

Calcola il rango di A usando la formula: $rk(A) = r$ tale che $\sigma_r > 0, \sigma_{r+1} = 0$, e confrontalo con l'output della funzione predefinita di **numpy**.

2 SVD per la compressione di immagini

Da un punto di vista computazionale, un'immagine in scala di grigi è una **matrice** di forma (altezza, larghezza), tale che l'elemento in posizione i, j contiene l'intensità del pixel nella posizione corrispondente. Un'immagine RGB è una tripla di matrici tale che, in posizione i, j , ciascuna delle tre matrici rappresenta la quantità di Rosso, Verde e Blu nel pixel corrispondente.

Per lavorare con le immagini, consideriamo il sottomodulo **skimage.data**:

```
import skimage

# Carica l'immagine "cameraman"
x = skimage.data.camera()

# Stampa la sua forma
print(f"Forma dell'immagine: {x.shape}.")
```

Forma dell'immagine: (512, 512).

Per visualizzare una matrice come immagine, si può usare la funzione **plt.imshow()** del modulo **matplotlib.pyplot**. Se l'immagine è in scala di grigi (come nel caso dell'immagine **camera** che stiamo considerando), è necessario impostare l'opzione **cmap='gray'** per visualizzarla come tale.

```
# Visualizza l'immagine
import matplotlib.pyplot as plt

plt.imshow(x, cmap="gray")
plt.show()
```

Nota

Figura del testo originale (non riprodotta qui): l'immagine "cameraman" in scala di grigi, 512×512 pixel, un classico esempio standard nell'elaborazione delle immagini.

Oltre alla visualizzazione, l'immagine resta comunque una matrice, e tutte le tecniche già studiate per le matrici possono essere usate sulle immagini. In particolare, l'approssimazione di rango k può essere usata per comprimerla con una perdita minima di informazione.

Ricordiamo che l'approssimazione di rango k , X_k , dell'immagine X è definita come

$$X_k = \sum_{i=1}^k \sigma_i u_i v_i^T$$

dove ogni σ_i è uno scalare, u_i è un vettore m -dimensionale, mentre v_i è un vettore n -dimensionale. Di conseguenza, il numero di valori richiesti per memorizzare X_k è $k(m + n + 1)$, mentre il numero di valori richiesti per memorizzare l'intera immagine X è mn . Di conseguenza, il fattore di compressione (cioè la percentuale di pixel risparmiati memorizzando X_k invece di X) può essere calcolato come:

$$c_k = 1 - \frac{k(m + n + 1)}{mn}.$$

Esercizio

Usando la funzione definita nell'esercizio precedente, calcola l'approssimazione di rango k dell'immagine cameraman X per diversi valori di k e osserva il comportamento di ciascuna ricostruzione. Inoltre, calcola e traccia il fattore di compressione c_k per ogni valore di k .

3 Riduzione della dimensionalità con la PCA

Lavorando con i dati, è comune avere accesso a informazioni non strutturate ad altissima dimensionalità (ad es. immagini, suoni, ...). Per lavorare efficacemente con esse, è necessario trovare un modo per proiettarle in uno spazio a bassa dimensionalità dove dati **semanticamente simili** siano **vicini**. Questo approccio è chiamato **riduzione della dimensionalità**.

Ad esempio, supponiamo che i nostri dati possano essere memorizzati in un array $N \times d$,

$$X = [x^{(1)}; x^{(2)}; \dots x^{(N)}] \in \mathbb{R}^{N \times d}$$

dove ogni punto dati $x^{(j)} \in \mathbb{R}^d$. L'idea delle tecniche di riduzione della dimensionalità in ML è trovare un operatore di proiezione $P: \mathbb{R}^d \rightarrow \mathbb{R}^k$, con $k \ll d$, tale che nello spazio proiettato $P(x)$, immagini semanticamente simili siano vicine. Se i punti in uno spazio proiettato formano popolazioni isolate tali che *all'interno* di ciascuna popolazione i punti sono vicini, mentre la distanza *tra* popolazioni è grande, li chiamiamo **cluster**. Un algoritmo di clustering è un algoritmo capace di trovare cluster a partire da dati ad alta dimensionalità.

3.1 Analisi delle Componenti Principali (PCA)

L'Analisi delle Componenti Principali (*Principal Component Analysis*, PCA) è probabilmente la tecnica più semplice, ma comunque efficace, per eseguire riduzione della dimensionalità e clustering. È un algoritmo non supervisionato, quindi non richiede alcuna etichetta. L'idea è la seguente: consideriamo un dataset $X \in \mathbb{R}^{N \times d}$ di dati ad alta dimensionalità e supponiamo di volerlo proiettare in uno spazio a bassa dimensionalità $\mathbb{R}^k$. Definiamo:

$$Z = [z^{(1)}; z^{(2)}; \dots z^{(N)}] \in \mathbb{R}^{N \times k},$$

la versione proiettata di X . Vogliamo trovare una matrice $P \in \mathbb{R}^{k \times d}$ tale che $Z = XP^T$, con il vincolo di mantenere nello spazio proiettato quanta più informazione possibile dai dati originali X .

Abbiamo già studiato che, quando si vuole proiettare una matrice mantenendo le informazioni, una buona idea è usare la sua Decomposizione ai Valori Singolari (SVD) e, in particolare, la SVD Troncata (*Truncated SVD*, TSVD). Sia $X \in \mathbb{R}^{N \times d}$, allora

$$X = U\Sigma V^T$$

è la SVD di X , dove $U \in \mathbb{R}^{N \times N}$, $V \in \mathbb{R}^{d \times d}$ sono matrici ortogonali ($U^T U = U U^T = I$ e $V V^T = V^T V = I$), mentre $\Sigma \in \mathbb{R}^{N \times d}$ è una matrice diagonale i cui elementi diagonali σ_i sono i valori singolari di X , in ordine decrescente ($\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_d$). Poiché i valori singolari rappresentano la *quantità di informazione* contenuta nei corrispondenti vettori singolari, mantenere i primi k valori singolari e vettori può essere la soluzione al nostro problema di proiezione. Infatti, dato $k < d$, definiamo la SVD Troncata di X come

$$X_k = U_k \Sigma_k V_k^T$$

dove $U_k \in \mathbb{R}^{N \times k}$, $\Sigma_k \in \mathbb{R}^{k \times k}$, e $V_k \in \mathbb{R}^{d \times k}$.

La PCA usa questa idea e definisce la matrice di proiezione come $P = V_k^T$, e di conseguenza,

$$Z = X P^T := X V_k$$

è lo spazio proiettato. Qui, le colonne di V_k sono chiamate **vettori di feature**, mentre le colonne di Z sono le **componenti principali** di X .

3.2 Implementazione

Per implementare la PCA, il primo passo è *centrare* i dati. Questo si fa calcolando il suo centroide $c(X)$, che viene poi sottratto a X per ottenere:

$$X_c = X - c(X),$$

che ha l'importante proprietà che $c(X_c) = 0$.

Nota

Dato un insieme $X = [x^{(1)}; x^{(2)}; \dots x^{(N)}]$, il suo **centroide** è definito come $c(X) = \frac{1}{N} \sum_{i=1}^N x^{(i)}$.

L'implementazione della PCA si legge quindi come segue:

- Consideriamo il dataset X ;
- Calcoliamo la versione centrata di X come $X_c = X - c(X)$, dove la sottrazione tra matrice e vettore viene eseguita *riga per riga*;
- Calcoliamo la SVD di X_c , $X_c = U \Sigma V^T$;
- Dato $k < n$, calcoliamo la SVD Troncata di X_c : $X_{c,k} = U_k \Sigma_k V_k^T$;
- Calcoliamo il dataset proiettato $Z_k = X_c V_k$;

3.3 Esempio in Python

Nel seguito, consideriamo come esempio il dataset MNIST, scaricabile da Kaggle. Per semplicità, l'ho rinominato `MNIST.csv` e l'ho posto in una cartella chiamata `data` dentro la cartella corrente del progetto.

kaggle.com/datasets/animatronbot/mnist-digit-recognizer

Nota

A causa di limitazioni di `github`, nel seguito consideriamo una versione più piccola dei dati, con dimensione 12486 invece di 42000. Tuttavia, manteniamo la discussione come se fosse usato il dataset completo, poiché è quanto ci si aspetta usando il dataset MNIST.

```
import numpy as np

# Carica i dati in memoria (usiamo numpy perche' pandas potrebbe non funzionare sui PC del Lab)
data = np.genfromtxt("data/MNIST.csv", delimiter=",")[1:]
print(f"Forma dei dati: {data.shape}")
```

Forma dei dati: (12486, 785)

Si noti che il nostro dataset è una tabella 42000×785 , dove le colonne **dalla seconda all'ultima sono i pixel** di un'immagine che rappresenta una cifra scritta a mano, mentre la **prima colonna** è il *target*, cioè l'intero che descrive la cifra rappresentata.

Suddividiamo quindi questi dati in dati di ingresso X e dati target Y .

```
# Suddivide i dati in una matrice X e un vettore Y dove:
#
# X ha dimensione (42000, 784)
# Y ha dimensione (42000, )
# Y e' la prima colonna di data, mentre X e' tutto il resto
X = data[:, :-1]
Y = data[:, -1]

print(f"Forma di X: {X.shape}, Forma di Y:{Y.shape}.")
```

Forma di X: (12486, 784), Forma di Y:(12486,).

3.4 Visualizzare le cifre

Abbiamo già osservato che X è un dataset di immagini che rappresentano cifre scritte a mano. Per visualizzarle, notiamo che nella documentazione si legge che ogni punto dati è un'immagine 28×28 in scala di grigi, che è stata **appiattita** (*flattened*). L'appiattimento è l'operazione di prendere un array bidimensionale (una matrice) e convertirlo in un array monodimensionale, concatenandone le righe. Questo può essere implementato in `numpy` con la funzione `a.flatten()`, dove `a` è un array `numpy` bidimensionale.

Poiché sappiamo che la dimensione di ogni immagine era 28×28 prima dell'appiattimento, possiamo invertire questa procedura **rimodellandole** (*reshaping*). Dopodiché, possiamo semplicemente visualizzarle con la funzione `plt.imshow()` di `matplotlib`, impostando `cmap` su `'gray'` dato che le immagini sono in scala di grigi.

```
import matplotlib.pyplot as plt

def visualize(X, idx):
    # Visualizza l'immagine di indice 'idx' dal dataset 'X'

    # Carica un'immagine in memoria
    img = X[idx]

    # La rimodella
    img = np.reshape(img, (28, 28))

    # Visualizza
    plt.imshow(img, cmap='gray')
    plt.show()

# Visualizza l'immagine numero 10 e la cifra corrispondente.
idx = 10
visualize(X, idx)
print(f"La cifra associata e': {Y[idx]}")
```

Nota

Figura del testo originale (non riprodotta qui): l'immagine 28×28 in scala di grigi, ricostruita dal vettore appiattito, corrispondente all'indice 10 del dataset.

La cifra associata e': 3.0

3.5 Filtrare le cifre

Per semplificare il compito di riduzione della dimensionalità fornito dalla PCA, possiamo filtrare solo alcune cifre dal dataset MNIST. Questo si può fare semplicemente definendo un `ndarray` booleano il cui valore in posizione i è `True` se e solo se la cifra nell'indice corrispondente rappresenta una delle cifre selezionate.

Nel seguito, mostriamo come questo si possa implementare per le cifre 3 e 4.

```
# Definisce l'array booleano per filtrare le cifre
filter_3or4 = (Y==3) | (Y==4)

# Definisce i dati filtrati
X_filtered = X[filter_3or4]
Y_filtered = Y[filter_3or4]

# Stampa la forma finale dei dati
print(f"Forma finale di X: {X_filtered.shape}. Forma finale di Y: {Y_filtered.shape}")

# Memorizza la forma di X nelle variabili N e d
N, d = X_filtered.shape

# Visualizza un campione solo per dimostrare che abbiamo solo 3 o 4
visualize(X_filtered, 10)
```

```
Forma finale di X: (8423, 784). Forma finale di Y: (8423,)
```

Nota

Figura del testo originale (non riprodotta qui): l'immagine di verifica dopo il filtraggio, che mostra una cifra 3 o 4 dal sottoinsieme filtrato.

3.6 Suddivisione del dataset

Prima di implementare l'algoritmo che esegue la PCA, si dovrebbe **sempre** suddividere il dataset in training set e test set. Ricordiamo che, per evitare qualsiasi possibile bias nel dataset suddiviso, la divisione dovrebbe essere fatta **casualmente**, mantenendo una proporzione di *almeno* 80% per il training set e 20% per il test set.

L'algoritmo che consideriamo per suddividere i dati è concettualmente molto simile a quello usato per campionare i batch nella Discesa del Gradiente Stocastica.

```
def split_data(X, Y, N_train):
    N, d = X.shape

    # Campiona gli indici e li mescola
    idx = np.arange(N)
    np.random.shuffle(idx)

    # Estrae l'insieme di indici
    train_idx = idx[:N_train] # per il training set
    test_idx = idx[N_train:] # e per il test set

    # Suddivisione di training
    X_train = X[train_idx]
    Y_train = Y[train_idx]

    # Suddivisione di test
    X_test = X[test_idx]
    Y_test = Y[test_idx]

    return (X_train, Y_train), (X_test, Y_test)
```



```
# Suddividere
N_train = int(N * 0.8)
(X_train, Y_train), (X_test, Y_test) = split_data(X_filtered, Y_filtered, N_train)

print(X_train.shape, X_test.shape)
```

```
(6738, 784) (1685, 784)
```

Ora il dataset (X, Y) è diviso nelle componenti di train e test, e possiamo implementare l'algoritmo PCA su X_{train} . **Ricordiamoci di non accedere mai a X_{test} durante il training.**

```
# Calcola il centroide
cX = np.mean(X_train, axis=0)

# Lo rende un vettore riga
cX = np.reshape(cX, (1, d))
print(f"Forma di c(X): {cX.shape}.")

# Centra i dati (riga per riga)
Xc = X_train - cX

# Calcola la decomposizione SVD
U, s, VT = np.linalg.svd(Xc, full_matrices=False)

# Dato k, calcola la SVD ridotta
k = 2
VTk = VT[:k]

# Definisce la matrice di proiezione
P = VTk

# Proietta X_train -> Z_train
Z_train = Xc @ P.T
print(f"Forma dei dati proiettati: {Z_train.shape}.")
```

```
Forma di c(X): (1, 784).
```

```
Forma dei dati proiettati: (6738, 2).
```

Attenzione

Quando non è richiesta la decomposizione SVD completa (come nel caso della PCA), è possibile calcolare la decomposizione SVD (ridotta) di una matrice X con `np.linalg.svd(X, full_matrices=False)`, per risparmiare tempo di calcolo.

3.7 Visualizzare i cluster

Quando $k = 2$, è possibile visualizzare i cluster in Python. In particolare, vogliamo tracciare i punti dati, con il colore della classe corrispondente, per verificare quanto bene l'algoritmo di clustering abbia funzionato in 2 dimensioni. Questo si può fare con la funzione `plt.scatter` di `matplotlib`. In particolare, se $Z_{train} = [z^{(1)}; z^{(2)}; \dots; z^{(N)}] \in \mathbb{R}^{N \times 2}$ è il dataset proiettato e $Y_{train} \in \mathbb{R}^N$ è il vettore delle classi corrispondenti, allora Z_{train} può essere visualizzato come:

```
# Visualizza i cluster
ax = plt.scatter(Z_train[:, 0], Z_train[:, 1], c=Y_train)
plt.legend(*ax.legend_elements(), title="Cifra") # Aggiunge alla legenda la lista delle cifre
plt.xlabel(r"$z_1$")
plt.ylabel(r"$z_2$")
plt.title("Proiezione PCA delle cifre MNIST 3 e 4")
plt.grid()
plt.show()
```

Nota

Figura del testo originale (non riprodotta qui): uno scatter plot bidimensionale (z_1 vs z_2) che mostra i punti proiettati delle cifre 3 e 4, colorati per classe, con i due cluster visibilmente separati.

Esercizio

Prova diverse combinazioni di cifre oltre a 3 o 4. Cosa osservi confrontando la separazione dei cluster e la somiglianza delle cifre?

Esercizio

Tenendo **fissa** la matrice di proiezione P , prova a proiettare il test set X_{test} come $Z_{test} = X_{test}P^T$. Visualizza Z_{test} sullo stesso scatterplot della proiezione del training set. Cosa osservi? Cosa succede se le cifre in X_{test} sono diverse dalle cifre in X_{train} ?

*Traduzione italiana del materiale del corso “Statistical and Mathematical Methods for Machine Learning”
(pagina originale in inglese).*

Materiale originale di Davide Evangelista — © 2023.